HARD BUDGETS EXPOSE THE REPAIR–BREAKAGE TRADEOFF IN AGENT WORKFLOWS

Anonymous authors
Paper under double-blind review

ABSTRACT

Whether structured agent workflows—generate, verify, refine—outperform a single model call is actively disputed: some studies report that multi-step structure dominates the compute–accuracy frontier, others that the advantage disappears once inference compute is genuinely equalised. We show that both findings are correct, and that the dispute persists because the field reports a single net number where two opposing effects are at work. Under a reservation-based ledger that enforces per-instance caps on tokens, calls, tool uses, latency and dollars *before* each call is issued, we decompose a workflow's effect into *repair*, the rate at which it rescues tasks the baseline fails, and *breakage*, the rate at which it destroys tasks the baseline already solves. On frozen test splits a standard verify–refine workflow repairs 20.5% of failures and breaks 14.0% of successes, netting -3.3 points while spending $5.5\times$ the dollars. The decomposition is an exact identity rather than a fitted model, and it yields a design rule that we then verify causally: protecting the incumbent answer—never overwriting a temperature-zero draft unless a verifier rejects it—drives breakage to 0.000 in all six family$\times$budget cells and converts a net-zero workflow into a small but reliable gain. Withholding the verifier's signal moves the tradeoff in the direction the identity predicts, costing 11.8 points. An automated search over a restricted workflow language, run to check that incumbent protection is not an artifact of our own design intuitions, rediscovers the same shape unprompted. We report every preregistered claim as it landed, including one refuted claim, three unsupported ones, and a locked out-of-domain prediction that failed because of a confound in our own experiment design.

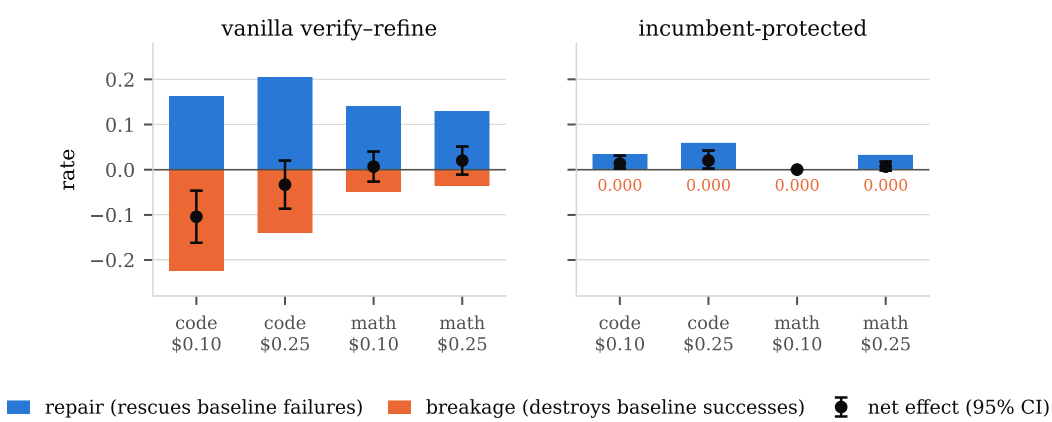

Figure 1: The same experiment, read two ways. **Left:** a standard verify–refine workflow repairs many tasks the baseline fails (blue, above the axis) and destroys many the baseline solves (orange, below), so the net effect (black, 95% CI)—the quantity the literature reports—is small and can take either sign. **Right:** protecting the incumbent answer removes the lower term entirely and the same machinery becomes a small, reliable gain. Frozen test splits, three execution seeds, matched hard budgets; panels share an axis.

1 INTRODUCTION

A large fraction of deployed language-model systems are not single calls but workflows: a model drafts an answer, a verifier inspects it, a refiner revises it, and the loop repeats until something stops it. The pattern is old enough to have a canon—self-refinement (Madaan et al., 2023), verbal reinforcement (Shinn et al., 2023), multi-agent debate (Du et al., 2024), self-consistency (Wang et al., 2023)—and new enough that its value is still contested. Recent work reports that multi-agent structure improves the compute–accuracy frontier under matched budgets (Anonymous, 2026b); other recent work argues the opposite, that once inference compute is genuinely equalised the advantage evaporates and reported gains reflect unaccounted computation rather than architecture. Both camps run careful experiments. Both report a single number: the difference in accuracy between a workflow and its baseline.

We think that number is the problem. A workflow that adds verification and refinement does two things at once, and they point in opposite directions. It rescues tasks the baseline would have failed—the reason anyone builds such systems. It also damages tasks the baseline had already solved, because a refiner asked to improve a correct answer will sometimes take it apart. Nothing in the standard evaluation separates these. When the two are of similar magnitude, their difference is small, unstable, and sensitive to conditions that vary between papers: how much budget each call is allowed, how reliable the verifier is, how strong the first draft was. A field that reports only the difference will produce exactly the pattern we observe—careful studies that disagree.

This paper separates them. We evaluate workflows under a reservation-based budget ledger: before any node executes, an upper bound on its billable cost is reserved against the remaining per-instance caps, and a node that cannot reserve does not run. Overruns become impossible rather than merely reported, which is what makes "compute-matched" mean something. Within that protocol we measure, for every workflow and every budget, not one number but three: the repair rate on tasks the baseline fails, the breakage rate on tasks the baseline solves, and the net effect the two produce.

Figure 1 shows what this reveals. On a program-synthesis benchmark at a \$0.25 per-task budget, a three-iteration verify–refine workflow repairs 20.5% of the baseline's failures—a real and substantial capability—while breaking 14.0% of its successes. The net is -3.3 points, at $5.5\times$ the cost. Tighten the budget to \$0.10 and the same workflow becomes significantly harmful (-10.4 points): with fewer affordable iterations, repair falls and breakage rises. Withhold the verifier's signal entirely and the effect falls to -11.8 points, because a refiner with nothing to gate on rewrites correct answers along with incorrect ones. Each of these is the movement of one term in the decomposition, produced by a controlled manipulation of one variable.

The decomposition is not a model we fit; for binary outcomes it is an algebraic identity, and we are explicit about that because it determines what the identity can be used for. It cannot be validated on the data used to estimate its terms. What it does buy is exhaustiveness—every effect a workflow has on accuracy is repair or breakage, and there is no third channel—and, rearranged, a design rule: refinement pays exactly when the breakage-to-repair ratio falls below $(1-p)/p$, where p is baseline accuracy. Because the ratio is a design choice, the rule is actionable. *Incumbent protection*—emit a temperature-zero draft and replace it only when a verifier actively rejects it—drives breakage to 0.000 in all six of our in-domain family$\times$budget cells, and the identical refinement machinery that was net-negative becomes a small, significant and considerably cheaper gain.

A design principle derived by us from our own decomposition and then tested by us is a route by which an intuition can be mistaken for a finding. We therefore built a budget-safe automated search over a restricted workflow language—not to propose a new optimiser, but to ask whether a procedure with no access to our reasoning arrives at the same place. In the code family it does: every search arm converges on a temperature-zero draft behind a verifier gate, and a single policy conditioned on remaining budget matches per-budget static search, at both searched budgets and at an intermediate budget it never saw, using half the search cost. In the math family it does not, and we report that too.

We take the reliability of these claims as seriously as the claims themselves. Every confirmatory hypothesis was written, committed and git-tagged before the corresponding frozen split was executed (Nosek et al., 2018), and we report all of them as they landed: four Part-I claims supported and one refuted, three Part-II claims not supported as stated, and a locked out-of-domain prediction that failed—because, as §7 diagnoses, our own out-of-domain split had been built without visible tests, so it varied domain and verifier availability at once. An automated audit recomputes every numeric claim in this paper from raw per-task logs.

The practical consequence is short. Structure is not free and is not uniformly good: it has an entry fee below which it is harmful, and it requires a verifier with signal. Where those hold, the thing worth building is not more structure but a guarantee that structure cannot destroy what already worked.

2 Background and related work

Iterative refinement and its discontents. Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) established the generate–critique–revise loop as a default building block, and multi-agent debate (Du et al., 2024) and self-consistency (Wang et al., 2023) extended the idea to populations of samples. Sceptical results arrived quickly: Huang et al. (2024) showed that models cannot reliably self-correct reasoning without external feedback, and attributed apparent gains to oracle labels leaking into the correction signal. Our decomposition puts a quantity behind that critique. Self-correction without external signal is exactly the regime in which b is unconstrained, and we measure what it costs: -11.8 points when the verifier is stripped of information, with breakage rising from 0.140 to 0.234.

Automated workflow design. A body of work searches over agent structures rather than fixing them by hand. GPTSwarm (Zhuge et al., 2024) represents agents as optimisable graphs and jointly tunes node prompts and edges; ADAS (Hu et al., 2024) has a meta-agent write agent code; AFlow (Zhang et al., 2025b) searches code-level workflows with tree search; AgentSquare (Shang et al., 2024) searches a modular design space; EvoFlow (Zhang et al., 2025a) evolves a population of heterogeneous workflows under cost and accuracy objectives; EvoAgentX (Wang et al., 2025) packages several such optimisers. These methods optimise *within* a design space. Our question is prior to it: what determines whether any structure in such a space is worth its cost, and why careful comparisons disagree. Our searcher is deliberately ordinary, because its role here is falsification rather than performance.

Budget-aware inference. Li et al. (2026) condition inference-tree node selection on the remaining-resource ratio, and Anonymous (2026a) make workflow *search* cost-aware through prefix reuse and caching. The budget conditioning we search over acts on workflow control flow rather than on tree node selection, and our ledger enforces caps before execution rather than accounting for them afterwards—a distinction that matters, because a workflow prevented from overspending behaves differently from one that overspends and is billed for it.

Test-time compute allocation. For a single model the tradeoff between sampling more and refining longer is well studied: Snell et al. (2024) show the optimal allocation depends on problem difficulty and Anonymous (2025) on verification granularity. This is the closest antecedent to our budget floor. What it does not do, and what we add, is separate the two directions of a workflow's effect. The crossovers reported there are crossovers in a net quantity, and the mechanism we identify—that structure destroys correct answers at a rate governed by verifier specificity—is invisible to it.

Evaluation. We use MBPP+ and HumanEval+ (Austin et al., 2021; Chen et al., 2021) in the rigorous-test formulation of EvalPlus (Liu et al., 2023), MATH (Hendrycks et al., 2021) and BIG-Bench Hard (Suzgun et al., 2023); multi-fidelity screening follows successive halving (Jamieson & Talwalkar, 2016; Li et al., 2018), and intervals are stratified paired bootstrap (Efron & Tibshirani, 1994) with Holm correction (Holm, 1979).

3 Hard budgets and a reservation ledger

A deployment budget is a vector of caps on input tokens, output tokens, LLM calls, tool calls, wall-clock seconds and dollars. We use five profiles from $0.02 to $0.25 per task. Two are searched—`tight` ($0.10, four calls) and `loose` ($0.25, eight calls)—and `unseen` ($0.15) is touched only at final evaluation, enforced by an assertion in the search path that fails if any selection code reads it.

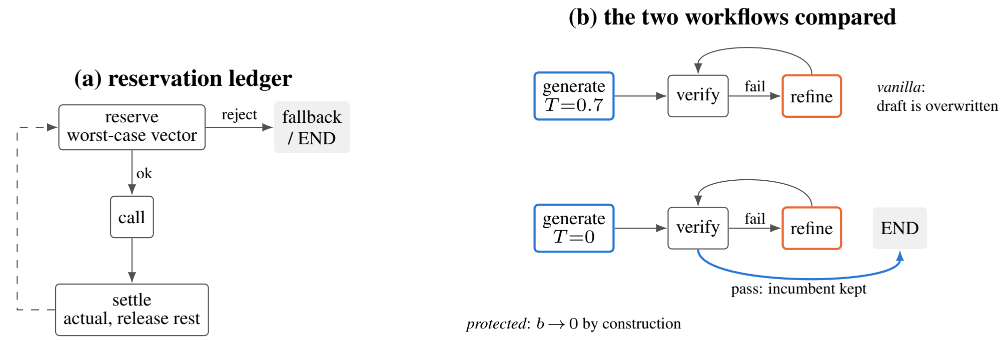

Figure 2: **(a)** Every metered call reserves an upper bound on its billable cost before executing; a node that cannot reserve does not run, and control follows a fallback edge. Overruns are prevented rather than recorded. **(b)** The only structural difference between the two workflows we compare throughout: the protected variant emits a temperature-zero draft and exits at the first verifier pass, so a correct incumbent is never handed to a refiner.

Before a node executes, an upper bound on its billable vector is computed and *reserved* against the remaining caps. If the reservation fails the node does not run: control follows a fallback edge or terminates with the current answer. After the call, actual usage is settled and the surplus released, and a settlement exceeding its own reservation is surfaced as a defect rather than absorbed. Concurrent calls, such as the k branches of a vote, reserve their full aggregate before any of them starts. Figure 2(a) shows the discipline and Appendix B proves the resulting invariant and reports a 3,000-workflow fuzz sweep with zero cap violations in any dimension.

This is not only hygiene: under post-hoc accounting a workflow that overspends is recorded as expensive, whereas under reservation it is prevented, and the prevention becomes an observable—reserve rejections turn out to be the visible face of the budget floor. Compute matching is enforced rather than audited.

Model. All experiments use a single frozen deployment of GPT-4o (Azure OpenAI, API version 2024-12-01-preview), fixed for the duration of the study; the deployment identifier and version are recorded with every trace. Unless a workflow node specifies otherwise, generation uses temperature 0.7 and refinement 0.7; the protected first draft uses temperature 0. No model is fine-tuned, and no results depend on model updates: the same deployment served every run reported here.

Verifiers. We distinguish two regimes and never conflate them. In the code family the workflow's verifier runs the task's own visible tests: environment feedback that is part of the task specification, with hidden extended tests reserved for the external grader. In the math and logic families no oracle is available to the workflow, and the verifier is a gold-free self-consistency check that re-derives the answer independently and compares. Gold answers never enter workflow state in any family.

Tasks. Code: MBPP+, split 120/40/150 for search, selection and confirmation, with HumanEval+ held out. Math: MATH levels 4–5 over four subjects, split 120/40/150, with two further subjects held out. Logic: BIG-Bench Hard, 120 tasks, reserved for §7 and used nowhere else. Splits were generated once with a fixed seed, hashed, and never regenerated. All final numbers use three execution seeds, seed-averaged per task, with stratified paired bootstrap intervals over 10,000 resamples.

4 THE REPAIR–BREAKAGE DECOMPOSITION

Let B be a baseline and W a workflow evaluated on the same tasks under the same budget. Write $p = \Pr[B \text{ correct}]$ and

$$r = \Pr[W \text{ correct} \mid B \text{ incorrect}], \qquad b = \Pr[W \text{ incorrect} \mid B \text{ correct}], \tag{1}$$

so that the net change in accuracy is

$$\Delta \;=\; \Pr[W] - \Pr[B] \;=\; (1-p)\,r \;-\; p\,b. \tag{2}$$

Table 1: Frozen test splits, 150 tasks per family, three execution seeds. Baselines are the strongest single-call structure per family (direct prompting for code, chain-of-thought for math). *Vanilla* is generate→verify→refine with up to three iterations; *protected* is the same loop with a temperature-zero incumbent replaced only on verifier rejection. Intervals are stratified paired bootstrap 95% CIs.

Family	Budget	Workflow	Δ (95% CI)	repair	breakage	\$/task
code	tight	vanilla	-0.104 $[-0.162, -0.047]$	0.162	0.224	0.0062
code	tight	protected	$+0.013$ $[+0.002, +0.031]$	0.034	**0.000**	0.0022
code	unseen	vanilla	-0.038 $[-0.093, +0.018]$	0.214	0.150	0.0085
code	unseen	protected	$+0.020$ $[+0.002, +0.042]$	0.060	**0.000**	0.0025
code	loose	vanilla	-0.033 $[-0.087, +0.020]$	0.205	0.140	0.0090
code	loose	protected	$+0.020$ $[+0.002, +0.042]$	0.060	**0.000**	0.0025
math	tight	vanilla	$+0.007$ $[-0.027, +0.040]$	0.140	0.050	0.0076
math	tight	protected	$+0.000$ $[+0.000, +0.000]$	0.000	**0.000**	0.0075
math	unseen	vanilla	$+0.013$ $[-0.020, +0.044]$	0.129	0.040	0.0141
math	unseen	protected	$+0.000$ $[-0.009, +0.009]$	0.022	**0.000**	0.0141
math	loose	vanilla	$+0.020$ $[-0.011, +0.051]$	0.129	0.037	0.0153
math	loose	protected	$+0.007$ $[-0.002, +0.018]$	0.032	**0.000**	0.0153

Equation 2 holds exactly, by the law of total probability, for binary per-task outcomes. We state this plainly because it determines what the equation is good for. It cannot be "validated" by estimating r and b on a dataset and checking that it reproduces Δ there; that check is tautological, and we do not present one. What it establishes is that the decomposition is *exhaustive*: every effect a workflow has on accuracy is repair or breakage. A literature that reports only Δ is reporting a difference of two quantities each of which can be an order of magnitude larger than the difference itself.

Rearranged, equation 2 gives a condition for when refinement is worth running:

$$\frac{b}{r} < \frac{1-p}{p}. \tag{3}$$

Two regimes follow. With an oracle verifier that never rejects a correct answer, $b \to 0$ and refinement cannot hurt. With a verifier carrying no information every answer is flagged, b rises to the unconditional rate at which revision damages correct answers, and equation 3 fails whenever the baseline is strong—which, for a competent model on a standard benchmark, it is. The threshold is punishing: $p = 0.73$ demands $b/r < 0.38$ and $p = 0.92$ demands $b/r < 0.09$.

The rule is actionable because its left-hand side is a design choice. *Incumbent protection* is the constructive route to $b \approx 0$ without an oracle: emit a temperature-zero draft, gate it behind the verifier, and replace it only on active rejection. Sections 5 and 7 test whether this works, where it fails, and whether the parameters transfer. They do not transfer for unconstrained workflows; they do for constrained ones, which is a claim about our construction rather than a law.

5 Confirmatory experiments

Five claims were written, committed and tagged (`prereg-freeze-partI`) before any frozen test split was executed. We report all five.

The net effect is zero and the terms are not (C1, supported). At `loose`, vanilla verify–refine differs from its baseline by -0.033 $[-0.087, +0.020]$ in code and $+0.020$ $[-0.011, +0.051]$ in math, at $5.5\times$ and $2.2\times$ the cost. Neither interval excludes zero: the workflow buys nothing measurable at several times the price. The decomposition shows what is happening underneath—in code it repairs a fifth of the baseline's failures and destroys a seventh of its successes.

Structure has an entry fee (C2, supported). At `tight`, code vanilla is -0.104 $[-0.162, -0.047]$. Moving from `loose` to `tight`, repair falls from 0.205 to 0.162 while breakage rises from 0.140 to 0.224: with fewer affordable iterations the loop is likelier to leave a task mid-revision, having already discarded a working answer. Figure 3 shows both terms moving. The

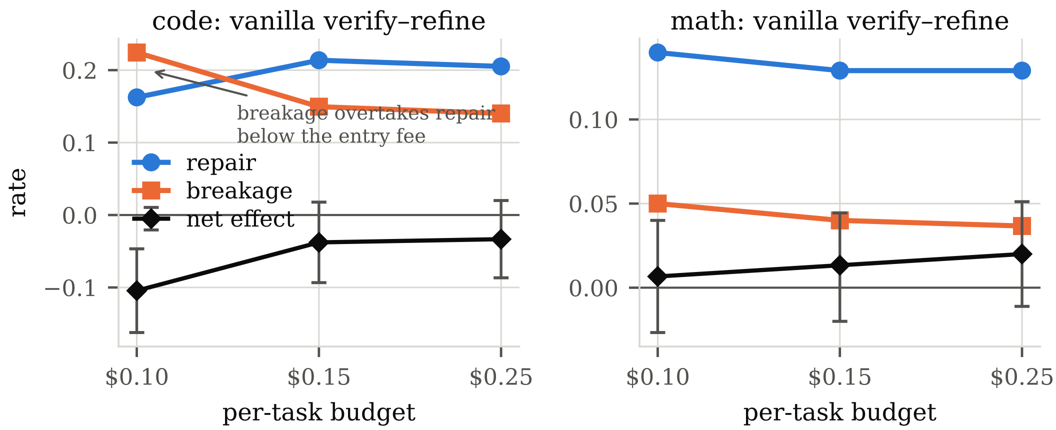

Figure 3: The budget floor is a movement of both terms at once. As the per-task budget tightens, the workflow can afford fewer refinement iterations: repair falls and breakage rises, and in the code family they cross, which is where the net effect turns significantly negative. Frozen test splits, 95% CIs on the net effect.

mechanism is visible in the ledger—35 of 150 tasks could not fund a planned refinement call. Below its entry fee, structure is not merely inefficient but harmful.

Breakage is controlled by the verifier (C3, supported). Masking visible tests at $1.0 \rightarrow 0.5 \rightarrow 0.0$ moves the code/`loose` effect monotonically: -0.033, -0.033, -0.118 $[-0.178, -0.058]$, with breakage climbing from 0.140 to 0.234. Because masking changes only the verifier this is a causal manipulation of the b term (Figure 4, left). It also supplies a mechanism for the sceptical literature: refinement without external signal does not merely fail to help, it destroys correct work.

Incumbent protection removes the lower term (C4, supported). Breakage is 0.000 in all six protected cells. In code the protected workflow is significantly positive at all three budgets ($+0.013$, $+0.020$, $+0.020$); The two workflows differ only as shown in Figure 2(b). In math it is non-inferior everywhere and positive but not significant at `loose` ($+0.007$ $[-0.002, +0.018]$). It is also cheaper than vanilla refinement—\$0.0025 against \$0.0090 per task at code/`loose` —because most tasks exit at the first verifier pass. The gain is modest, and that is the point: what changed is not how much the workflow repairs but that it no longer pays for repair with damage.

Repair is not bounded by verifier type (C5, refuted). We predicted that oracle-verified repair would strictly exceed non-oracle repair, with the math interval containing zero. Observed: code repair 0.060 $[0.000, 0.137]$, math repair 0.032 $[0.000, 0.075]$. The intervals overlap substantially. Gold-free self-consistency checking produces real repair, and we make no claim about verifier-type-bounded repair beyond these intervals.

6 AUTOMATED SEARCH AS A BIAS CHECK

Incumbent protection was derived by us, from our own decomposition, and tested by us. We therefore built a searcher whose purpose is to check whether an optimiser with no access to that reasoning arrives at the same construction.

Candidates are workflows in a restricted language: at most eight nodes, at most two loops of at most three iterations, a fixed node whitelist, no model-generated executable code, and static validation before any spend (Appendix A). Search is evolutionary with multi-fidelity racing over nested task subsets (Jamieson & Talwalkar, 2016), evaluating every candidate jointly at both searched budgets and ranking by a score that averages per-budget accuracy with the worst budget, so that a candidate cannot win by specialising. The budget-contingent arm and the static control differ in exactly one respect—whether edges may condition on remaining budget—and we verified over 200 sampled

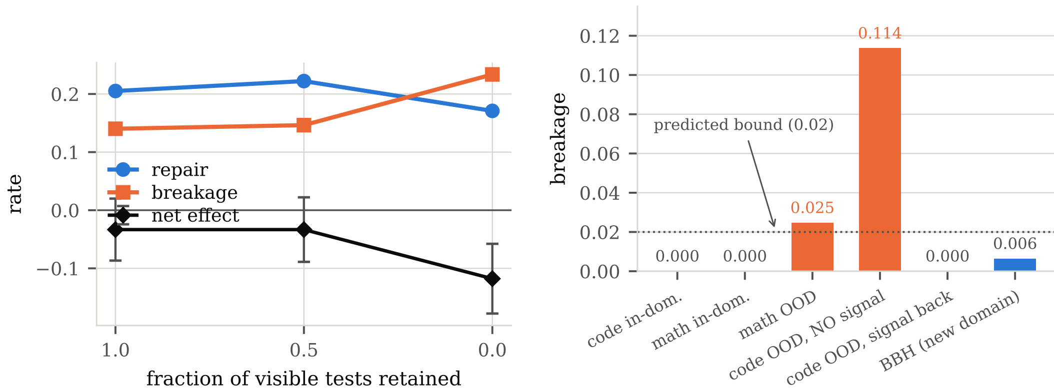

Figure 4: **Left:** withholding the verifier's signal is a causal manipulation of the breakage term. As visible tests are removed, breakage rises and repair falls, and the net effect drops to -0.118 $[-0.178, -0.058]$. **Right:** breakage of the incumbent-protecting construction across every condition we measured, including a third domain whose prediction was locked before execution. It stays at or near the predicted bound wherever the verifier carries signal and jumps to 0.114 in the single condition with none; restoring signal on the *same* tasks restores the guarantee.

workflows that the static arm never emits a budget predicate. Protocol A gives the static control a full search budget *per budget tier*, twice what the budget-contingent arm receives.

Convergence. At full fidelity on the search split, every arm—the budget-contingent one and both static controls—selected the same shape: a temperature-zero first draft, a verifier gate, and refinement only on rejection. The budget-contingent arm additionally gated refinement on remaining budget (threshold 0.6; one variant used a two-stage $0.3/0.6$). A procedure with no access to equation 2 reconstructed its prescription.

Confirmatory results, as they landed. Five claims were tagged (`prereg-freeze-partII`) before any searched policy touched validation or test. **P1** (non-inferiority at both budgets) is *not supported*: it holds in code at both budgets ($+0.013$, LCB -0.001; $+0.001$, LCB -0.010) and at math/`loose` ($+0.049$ $[+0.009, +0.091]$, significantly better), but fails at math/`tight` (-0.022, LCB -0.049), and P1 required all four cells. **P2** (a $\geq 20\%$ cost reduction) is *not supported*: code achieved 10.5%, and math was 191% more expensive, because the selected budget-contingent policy verifies and refines while the selected static policy is a bare generate—a Pareto trade, not a cost win. **P3** (the unseen \$0.15 budget) holds in code, where the policy is indistinguishable from the preregistered static transfer control (-0.001 $[-0.013, +0.010]$) at a budget no search ever saw, and fails in math (-0.011, LCB -0.040). **P4**: only 3 of 12 selected policies show the protected shape, against every arm on the search split; the preregistered one-shot selection uses a 40-task validation split whose variance is large relative to the 1–2 point gaps between candidates, and we report this as a protocol defect rather than repairing it after the freeze. **P5**: in code the budget-contingent search costs \$60 against \$120 for per-budget static search and deploys \$0.00029 per task cheaper, so it dominates from the first task; in math it does not.

Figure 5 in Appendix F shows the deployment comparison. The defensible readings are narrow. An independent optimiser reconstructs incumbent protection in the code family, which is evidence that the principle is not our design bias. One policy can cover several budgets, including an unseen one, at half the design cost, in the code family. Neither transfers to math.

7 External validity

A locked prediction that failed. Before computing any out-of-domain effect we committed (tag `ood-predictions-locked`) predictions for HumanEval+ and held-out math subjects, transferring r and b from the in-domain frozen split. The prediction failed: mean absolute error 0.061, sign agreement $6/8$, and the structural breakage bound violated in code at 0.125 and 0.114.

Table 2: Breakage of the incumbent-protecting workflow across verifier regimes and domains. The guarantee tracks verifier signal rather than domain: the one condition without signal is an order of magnitude worse than the rest, and restoring signal on the same tasks restores it.

Condition	Verifier	breakage	Δ (95% CI)
in-domain code (MBPP+)	oracle tests	0.000	$+0.020$ $[+0.002, +0.042]$
in-domain math (MATH)	non-oracle check	0.000	$+0.007$ $[-0.002, +0.018]$
out-of-domain math (held-out subjects)	non-oracle check	0.025	-0.003 $[-0.027, +0.017]$
out-of-domain code (HumanEval+)	*none*	0.114	-0.087 $[-0.140, -0.033]$
out-of-domain code, signal restored	recovered tests	0.000	$+0.005$ $[+0.000, +0.015]$
new domain (BIG-Bench Hard)	non-oracle check	0.006	$+0.025$ $[+0.003, +0.050]$

The confound was ours. Our out-of-domain code split had been constructed with empty visible tests, so its verifier ran in the no-signal regime while the transferred parameters came from the oracle regime: the test varied domain *and* verifier availability simultaneously. Substituting matched no-signal parameters does not rescue it either (errors 0.09–0.15), so the quantitative transfer claim fails on its own terms and we do not make it. We then built the condition we should have built in the first place: visible tests recovered from HumanEval+ docstring examples ($68/100$ tasks), differing from in-domain only in domain. Breakage returns to 0.000 (Table 2), and the transferred prediction is accurate for protected workflows (errors 0.002 and 0.000) while remaining badly wrong for vanilla ones (0.169, 0.108)—the constrained-versus-unconstrained asymmetry of §4.

A prospective test in a third domain. Everything above is retrospective: the domains were chosen and the framework then applied. To test it prospectively we chose a domain no part of this project had touched— BIG-Bench Hard logical deduction, shuffled-object tracking and date understanding (Suzgun et al., 2023), 120 multiple-choice tasks with exact-match grading and a gold-free verifier—wrote down what the framework predicts, committed it (`prospective-locked`), and only then ran anything. The predictions were structural rather than quantitative, since the parameters had already been shown not to transfer: **PV1**, protected breakage ≤ 0.02 at both budgets; **PV2**, vanilla breakage materially higher; **PV3**, protected effect ≥ -0.01; **PV4**, explicitly no prediction about repair magnitude.

PV1 holds in all four cells (0.000, 0.006, 0.000, 0.003) and PV3 in all four. This is a demanding regime: the baseline is right 92–94% of the time, so equation 3 demands $b/r < 0.09$ and even small breakage should flip the sign. PV2 does not hold—at `loose`, vanilla breakage is 0.006, no worse than protected. The reason is visible in the data and is a boundary we did not anticipate. Breakage requires that refinement be *able* to destroy a correct answer; with single-label answers and a 94% baseline there is little to damage, and at `tight`, where refinement acts under pressure, the gap reappears (0.026 against 0.000). Incumbent protection is insurance, and its premium is only visible where the risk is real.

8 LIMITATIONS

We state these at length, because several bound the claims directly.

One model. Every number comes from GPT-4o. Both p and r depend on model quality and b plausibly does too; nothing here establishes that the tradeoff transfers across model scales, and our own tests show the parameters are unstable even across domains for unconstrained workflows. A weaker model has a lower p, which by equation 3 loosens the threshold and should make structure easier to justify — a prediction we have not tested.

The identity is not a forecast. equation 2 is exact for binary outcomes. Its content is exhaustiveness plus the rule equation 3; it makes no prediction falsifiable on the data used to estimate its terms. The genuine transfer tests are those of §7, and one of them failed.

Two measurement defects in the released data. Two implementation faults, both fixed, left traces in the frozen-test results and we report their effect rather than only the fix. A pre-call token estimate that under-counted punctuation-dense prompts caused 4.7% of executions to settle above their own reservation and be scored as failures, unevenly across arms; Appendix D recomputes every headline contrast on the defect-free subset, where the breakage guarantee is unaffected and the harm effects grow slightly. And the symbolic-equivalence branch of our math grader never executed, because the library was absent from the environment that produced those runs, so 2.6% of math answers were misgraded; re-grading every stored response at no additional inference cost moved the math baseline from 0.700 to 0.733 and cost two math sub-claims their significance (Appendix E). Every number in this paper uses the corrected grading, and both gradings are retained in the released data.

Selection variance in Part II. Our preregistered rule selects a policy once on a 40-task validation split, which cannot resolve the 1–2 point gaps between candidates—hence shape convergence is universal in the search-split analysis but $3/12$ among selected policies. A larger validation split is the fix; we did not retrofit it after the freeze.

No mechanism for the math Part-II boundary. The budget-contingent policy is better at math/`loose` and worse at math/`tight` and we cannot explain this with data. It would be easy to present it as a boundary condition predicted by equation 3; we checked, and the measured ratio sits essentially on the threshold, so the rule is silent there and does not predict harm. We therefore do not claim it does.

Restricted design space, coarse verifier taxonomy. Results about "structure" are results about a space of at most eight nodes and two loops with a fixed whitelist, and we treat verifiers as oracle, non-oracle-with-signal or no-signal, whereas real verifiers vary continuously in sensitivity and specificity. equation 2 folds both into r and b; separating them would give a sharper rule and is the obvious next step.

9 CONCLUSION

Whether agent workflow structure helps has no budget-free answer, and reporting a net effect conceals why. Structure repairs and structure breaks; under matched hard budgets the two are of comparable size, so their difference is small, unstable, and reads differently depending on where in the budget range one measures. Once separated, the design question becomes concrete: make breakage impossible. Protecting the incumbent answer does that—breakage 0.000 across every in-domain cell and three domains—and it is what an independent optimiser converges to when asked to search the space itself. It fails precisely where the mechanism says it must: when the verifier has no signal to gate on.

REPRODUCIBILITY STATEMENT

All splits are hashed and frozen. Every confirmatory claim was git-tagged before the corresponding split was executed (`prereg-freeze-partI`, `prereg-freeze-partII`, `ood-predictions-locked`, `prospective-locked`). The anonymised repository contains the workflow language and its validator, the reservation ledger with its fuzz harness, every search registry with parent and mutation lineage, all raw per-task traces under both gradings, the preregistration with its append-only deviation log including the refuted claims, and a single command that recomputes every table and figure in this paper from raw logs and exits non-zero on any disagreement.

AI USE STATEMENT

Language models were used as coding assistants during implementation, and are the subject of study. All experimental design decisions, preregistered hypotheses, analyses and claims are the authors' own, and every reported number is recomputed from raw logs by the audit script in the repository.

REFERENCES

Anonymous. Rethinking optimal verification granularity for compute-efficient test-time scaling. *arXiv preprint arXiv:2505.11730*, 2025.

Anonymous. Agent-UCT: Upper confidence bounds applied to trees for agentic workflow optimization with cost-awareness. *arXiv preprint arXiv:2607.24162*, 2026a.

Anonymous. Multi-agent reasoning improves compute efficiency: Pareto-optimal test-time scaling. *arXiv preprint arXiv:2605.01566*, 2026b.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning*, 2024.

Bradley Efron and Robert J Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations*, 2024.

Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Artificial Intelligence and Statistics*, 2016.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

Y. Li et al. Spend less, reason better: Budget-aware value tree search for LLM agents. *arXiv preprint arXiv:2603.12634*, 2026.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

Brian A Nosek, Charles R Ebersole, Alexander C DeHaven, and David T Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics*, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX: An automated framework for evolving agentic workflows. In *Proceedings of EMNLP: System Demonstrations*, 2025.

Guibin Zhang, Kaijie Niu, Yusen Ma, Zhixun Yang, Jiaxuan Cheng, and Xiaojiang Wang. EvoFlow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373*, 2025a.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations*, 2025b.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *International Conference on Machine Learning*, 2024.

A Workflow language

A workflow is a JSON graph validated statically before any spend. Node whitelist: `generate`, `refine`, `vote` ($k \leq 5$), `verify`, `decompose`, `aggregate`, `branch`, `END`. Structural constraints: at most 8 nodes; at most 2 loops, each with `max_iter` ≤ 3; every node must reach `END` via non-loop edges, checked by reachability. Edge conditions come from a fixed grammar: `always`, `verify_passed`, `verify_failed`, `budget_below:`q, `budget_above:`q, with $q \in (0, 1)$ read against the ledger's most-binding remaining fraction. The meta-agent emits JSON patches only; no model-generated text is executed as program logic. Candidate solutions to code tasks are executed, but only inside a network-isolated subprocess with a 5 s CPU limit and a 4 GB address-space limit. Invalid candidates are rejected at zero API cost, and we report the invalid rate alongside search cost rather than hiding it.

B Budget safety

Proposition 1. *Fix a task, a budget profile C, and any workflow expressible in the language. If (i) every metered call is preceded by a successful reservation of an upper bound on its billable vector, (ii) concurrent calls reserve their full aggregate before any of them starts, and (iii) every settlement reports actual usage no greater than its reservation, then cumulative usage never exceeds C in any countable dimension.*

Proof sketch. Induct on the number of settled calls, with invariant *used* + *reserved* $\leq C$ componentwise. It holds initially. A reservation is admitted only if the invariant survives adding its vector to *reserved*; a settlement moves a quantity no larger than the reservation from *reserved* to *used* and releases the remainder, which cannot increase the sum. Rejected reservations leave the state unchanged and divert control flow, so no unreserved call occurs. $\square$

Wall-clock time is not reservable, since its consumption is not known in advance; it is metered continuously and enforced by admission checks plus per-call deadlines. Provider-side cancellation latency is reported separately from policy-attributable overruns. A fuzz sweep of 3,000 randomly generated legal workflows against randomly drawn budget profiles, with a mocked model, produced zero cap violations in any dimension.

Assumption (iii) is an assumption about the *estimator*, and ours was initially wrong: a characters-per-token heuristic under-counted punctuation-dense code prompts by roughly 55%, so some settlements exceeded their own reservations. The implementation surfaces these as defects rather than absorbing them, which is how we found them. Appendix D quantifies the residual effect on the frozen test results, which were collected before the estimator was replaced with an exact tokeniser count plus a 5% margin.

C Failure taxonomy

Categories were fixed before inspection. Counts are over all failures in the frozen test split at seed 0, pooled across structures and budgets.

Category	share	example
first draft wrong, no refinement triggered	28.4%	`direct`, mbpp_103
premature budget exhaustion (reserve rejected)	26.3%	protected math, algebra_1071
verifier passed a wrong answer	19.8%	protected code, mbpp_113
ledger defect (settle overrun; see D)	18.4%	protected code, mbpp_578
refined but still wrong (signal insufficient)	7.1%	protected code, mbpp_160

The third row is the mechanism behind b: a verifier that accepts an incorrect answer cannot trigger the repair that would fix it, and one that rejects a correct answer invites the breakage that destroys it.

D Sensitivity to the ledger defect

4.7% of frozen-test executions ($466/9{,}900$) terminated with a settlement overrun, unevenly across arms (protected 6.5%, vanilla 4.5%, baseline 3.6%), and were scored as failures. We recompute every headline contrast on the subset of tasks where *neither* arm hit the defect in *any* seed.

Contrast	all tasks	defect-free subset	n
C1 code loose, vanilla − direct	-0.033 $[-0.087, +0.020]$	-0.046 $[-0.096, +0.005]$	146
C2 code tight, vanilla − direct	-0.104 $[-0.162, -0.047]$	-0.115 $[-0.173, -0.056]$	148
C3 code loose, no signal	-0.118 $[-0.178, -0.058]$	-0.132 $[-0.192, -0.074]$	144
C4 code tight, protected	$+0.013$ $[+0.002, +0.031]$	$+0.014$ $[+0.002, +0.032]$	147
C4 code loose, protected	$+0.020$ $[+0.002, +0.042]$	$+0.018$ $[+0.002, +0.041]$	145
C4 code unseen, protected	$+0.020$ $[+0.002, +0.042]$	$+0.021$ $[+0.002, +0.043]$	146

Breakage of the protected construction is 0.000 under both views in every cell, so C4's structural claim is unaffected, and the C1–C3 harm effects become slightly *larger* on the defect-free subset, so those conclusions are not artifacts of the defect.

E Math re-grading

Our math grader compares a boxed answer by string match, then numerically, then by symbolic equivalence. The third stage never fired: the symbolic library was absent from the environment that produced the frozen runs, and the import sits inside an exception handler. Because every model response is stored, re-grading required no additional inference. 623 of $24{,}334$ stored math answers (2.6%) changed grade, almost all of them formatting variants of the same value. The math baseline moved from 0.700 to 0.733. Two math sub-claims lost significance (protected at `unseen` and at `loose`); no verdict changed, and breakage remained 0.000 in every in-domain protected cell. All numbers in this paper use the corrected grading; the original field is preserved on disk.

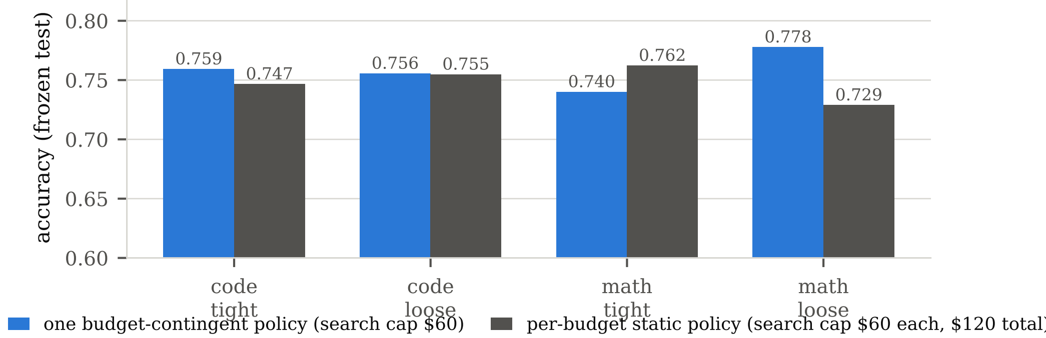

Figure 5: Protocol A on the frozen test split: one budget-contingent policy against the static policy searched specifically for each budget, which received twice the total search budget. We deliberately do not plot the two search procedures' internal scores against each other—the budget-contingent score is a cross-budget minimum and the static score is a single-budget value, so one axis would compare incomparable quantities. Deployment accuracy at a given budget is comparable, and is what is shown. Code figures average three search seeds.

F SEARCH DETAILS

A note on the racing rule. Our first implementation compared a candidate's lower confidence bound against the incumbent's with a fixed slack of 0.10. The Hoeffding half-width (Hoeffding, 1963) differs by 0.107 between the n=24 and n=64 rungs, so promotion from the cheapest rung was arithmetically impossible and almost no candidate ever left it. Whenever rung sizes differ, racing must compare a candidate's *upper* bound against the incumbent's lower bound; a fixed slack cannot substitute for the difference in interval widths. No result reported in this paper comes from runs made before this was corrected.

Fidelity rungs are nested prefixes of the search split: 24, 64 and 120 tasks, with 1, 1 and 2 execution seeds. Racing eliminates a candidate only when its Hoeffding upper bound falls below the incumbent's lower bound at the same or a higher rung. Selection and archive ranking use a fidelity-comparable mean-based score; the conservative bound is used only for the racing rule.

Run (code, search seed 0)	candidates	spend	rung distribution	archive
budget-contingent	33	$63.0	9/5/19	11
static @ tight	76	$60.7	35/5/36	38
static @ loose	30	$61.6	5/5/20	8

G COST ACCOUNTING

All per-task dollar figures are *logical* costs: what the workflow would cost without response caching, which is the quantity relevant to deployment. Search-phase caching reduced the actual invoice below this figure; final evaluation runs were executed with caching disabled. Summed over every experiment reported here, logical spend was $1,317.74 against a global hard cap of $2,000 enforced by the ledger's circuit breaker.

H PREREGISTRATION AND OUTCOMES

The repository contains the preregistration with an append-only deviation log. Claims were committed and git-tagged before the corresponding frozen split was executed. Outcomes as recorded: Part I C1–C4 supported and C5 refuted; Part II P1–P3 not supported as stated, P4 weaker than on the search split, P5 reported; the locked out-of-domain transfer prediction failed; the prospective predictions PV1 and PV3 held and PV2 did not.